

Exercícios de autorrevisão

- 8.1** Escreva uma única instrução que possibilite a execução das tarefas a seguir. Suponha que as variáveis `c` (que armazena um caractere), `x`, `y` e `z` sejam do tipo `int`, as variáveis `d`, `e` e `f` sejam do tipo `double`, a variável `ptr` seja do tipo `char *` e os arrays `s1[100]` e `s2[100]` sejam do tipo `char`.
- Converta o caractere armazenado na variável `c` em uma letra maiúscula. Atribua o resultado à variável `c`.
 - Determine se o valor da variável `c` é um dígito. Use o operador condicional conforme mostrado nas figuras 8.2 a 8.4 para imprimir “ é um “ ou “ não é um “ quando o resultado for exibido.
 - Converta a string “1234567” em `long` e imprima o valor obtido.
 - Determine se o valor da variável `c` é um caractere de controle. Use o operador condicional para imprimir “ é um “ ou “ não é um “ quando o resultado for exibido.
 - Leia uma linha de texto para o array `s1` a partir do teclado. Não use `scanf`.
 - Imprima a linha de texto armazenada no array `s1`. Não use `printf`.
 - Atribua a `ptr` o local da última ocorrência de `c` em `s1`.
 - Imprima o valor da variável `c`. Não use `printf`.
 - Converta a string “8.63582” em `double` e imprima o valor.
 - Determine se o valor de `c` é uma letra. Use o operador condicional para imprimir “ é uma ” ou “ não é uma ” quando o resultado for exibido.
 - Leia um caractere do teclado e armazene o caractere na variável `c`.
 - Atribua a `ptr` o local da primeira ocorrência de `s2` em `s1`.
 - Determine se o valor da variável `c` é um caractere imprimível. Use o operador condicional para imprimir “ é um “ ou “ não é um ” quando o resultado for exibido.
 - Leia três valores `double` para as variáveis `d`, `e` e `f` a partir da string “1.27 10.3 9.432”.
 - Copie a string armazenada no array `s2` para o array `s1`.
 - Atribua a `ptr` o local da primeira ocorrência em `s1` de qualquer caractere de `s2`.
 - Compare a string em `s1` com a string em `s2`. Imprima o resultado.
 - Atribua a `ptr` o local da primeira ocorrência de `c` em `s1`.
 - Use `sprintf` para imprimir os valores das variáveis inteiras `x`, `y` e `z` no array `s1`. Cada valor deve ser impresso com uma largura de campo 7.
 - Acrescente 10 caracteres da string em `s2` à string em `s1`.
 - Determine o comprimento da string em `s1`. Imprima o resultado.
 - Converta a string “-21” em `int` e imprima o valor.
 - Atribua a `ptr` o local do primeiro token em `s2`. Os tokens na string `s2` são separados por vírgulas (,).
- 8.2** Mostre dois métodos diferentes de inicializar o array de caracteres `vogal` com a string de vogais “AEIOU”.
- 8.3** O que é impresso quando as instruções em C, a seguir, são executadas? Se a instrução contiver um erro, descreva o erro e mostre como corrigi-lo. Considere as seguintes declarações de variável:
- ```
char s1[50] = “jack”, s2[50] = “ jill”,
s3[50], *sptr;
```
- `printf( “%C%S”, toupper( s1[ 0 ] ), &s1[ 1 ] );`
  - `printf( “%s”, strcpy( s3, s2 ) );`
  - `printf( “%s”, strcat( strcat( strcpy( s3, s1 ), “ and “ ), s2 ) );`
  - `printf( “%u”, strlen( s1 ) + strlen( s2 ) );`
  - `printf( “%u”, strlen( s3 ) );`
- 8.4** Encontre o erro em cada um dos segmentos de programa a seguir e explique como corrigi-lo:
- ```
char s[ 10 ];
strncpy( s, “olá”, 5 );
printf( “%s\n”, s );
```
 - ```
printf(“%s”, ‘a’);
```
  - ```
char s[ 12 ];
strcpy( s, “Bem-vindo” );
```
 - ```
if (strcmp(string1, string2)) {
 printf(“As strings são iguais\n”);
}
```

## ■ Respostas dos exercícios de autorrevisão

- 8.1**
- a) `c = toupper( c );`
  - b) `printf( “%c’%sdigit\n”, c, isdigit( c ) ? “ é um “ : “ não é um “ );`
  - c) `printf( “%ld\n”, atol( “1234567” ) );`
  - d) `printf( “%c’%scaractere de controle\n”, c, iscntrl( c ) ? “ é um “ : “ não é um “ );`
  - e) `fgets( s1, 100, stdin );`
  - f) `puts( s1 );`
  - g) `ptr = strrchr( s1, c );`
  - h) `putchar( c );`
  - i) `printf( “%f\n”, atof( “8.63582” ) );`
  - j) `printf( “%c’%sletra\n”, c, isalpha( c ) ? “ é uma “ : “ não é uma “ );`
  - k) `c = getchar();`
  - l) `ptr = strstr( s1, s2 );`
  - m) `printf( “%c’%scaractere imprimível\n”, c, isprint( c ) ? “ é um “ : “ não é um “ );`
  - n) `sscanf( “1.27 10.3 9.432”, “%f%f%f”, &d, &e, &f );`
  - o) `strcpy( s1, s2 );`
  - p) `ptr = strpbrk( s1, s2 );`
  - q) `printf( “strcmp( s1, s2 ) = %d\n”, strcmp( s1, s2 ) );`
  - r) `ptr = strchr( s1, c );`
  - s) `sprintf( s1, “%7d%7d%7d”, x, y, z );`
  - t) `strncat( s1, s2, 10 );`
  - u) `printf( “strlen(s1) = %u\n”, strlen( s1 ) );`
  - v) `printf( “%d\n”, atoi( “-21” ) ); *`
  - w) `ptr = strtok( s2, “,” );`
- 8.2**
- ```
char vogal[] = “AEIOU”;
char vogal[] = { ‘A’, ‘E’, ‘I’, ‘O’, ‘U’, ‘\0’ };
```
- 8.3**
- a) Jack
 - b) jill
 - c) jack e jill
 - d) 8
 - e) 13
- 8.4**
- a) Erro: A função `strncpy` não escreve um caractere nulo de finalização no array `s`, pois seu terceiro argumento é igual ao comprimento da string “hello”.
Correção: Mude o terceiro argumento de `strncpy` para 6, ou atribua ‘\0’ a `s[ 5 ]`.
 - b) Erro: A tentativa de imprimir uma constante de caractere como uma string.
Correção: Use `%c` para exibir o caractere ou substitua ‘a’ por “a”.
 - c) Erro: O array de caracteres `s` não é grande o suficiente para armazenar o caractere nulo de finalização.
Correção: Declare o array com mais elementos.
 - d) Erro: A função `strcmp` retorna 0 se as strings forem iguais; portanto, a condição na estrutura `if` é falsa, e o `printf` não será executado.
Correção: Compare o resultado de `strcmp` com 0 na condição.

■ Exercícios

- 8.5** **Teste de caracteres.** Escreva um programa que receba um caractere do teclado e teste-o com cada uma das funções na biblioteca de tratamento de caractere. O programa deverá imprimir o valor retornado por cada função.
- 8.6** **Exibição de strings em maiúsculas e minúsculas.** Escreva um programa que receba uma linha de texto no array de char `s[100]`. Imprima a linha com letras maiúsculas e minúsculas.
- 8.7** **Conversão de strings em inteiros na realização de cálculos.** Escreva um programa que receba quatro strings que representem inteiros, converta as strings em inteiros, some os valores e imprima o total dos quatro valores.
- 8.8** **Conversão de strings em ponto flutuante na realização de cálculos.** Escreva um programa que receba quatro strings que representem valores de ponto flutuante, converta as strings em valores `double`, some os valores e imprima o total dos quatro valores.
- 8.9** **Comparação de strings.** Escreva um programa que use a função `strcmp` para comparar duas strings digitadas pelo usuário. O programa deverá indicar se a primeira string é menor, igual ou maior que a segunda.

8.10 Comparação de partes de strings. Escreva um programa que use a função `strncmp` para comparar duas strings digitadas pelo usuário. O programa deverá receber o número de caracteres a ser comparado, depois informar se a primeira string é menor, igual ou maior do que a segunda.

8.11 Sentenças aleatórias. Escreva um programa que use a geração de números aleatórios na criação de sentenças. O programa deverá usar quatro arrays de ponteiros para char chamados `artigo`, `substantivo`, `verbo` e `preposicao`. O programa deverá criar uma sentença, selecionando uma palavra aleatória de cada array na seguinte ordem: `artigo`, `substantivo`, `verbo`, `preposicao`, `artigo` e `substantivo`. À medida que cada palavra é obtida, ela deve ser concatenada com as palavras anteriores em um array grande o suficiente para conter a sentença inteira. As palavras devem ser separadas por espaços. Quando a sentença final for exibida, ela deverá começar com uma letra maiúscula e terminar com um ponto. O programa deve gerar 20 dessas sentenças. Os arrays devem ser preenchidos da seguinte forma: O array `artigo` deverá conter os artigos “o”, “a”, “um”, “uma”, “algum” e “alguma”; o array `substantivo` deverá conter os nomes “menino”, “menina”, “cão”, “cidade” e “carro”; o array `verbo` deverá conter os verbos “dirigiu”, “saltou”, “correu”, “caminhou” e “saltou”; o array `preposicao` deverá conter as preposições “para”, “de”, “sobre”, “sob” e “em”.

Depois que o programa for escrito e estiver em funcionamento, modifique-o para que ele produza uma pequena história que consista em várias dessas sentenças. (Que tal a possibilidade de um escritor de tese aleatória?)

8.12 Limericks. Uma limerick é um poema humorístico de cinco linhas, em que a primeira e a segunda linhas rimam com a quinta, e a terceira linha rima com a quarta. Usando técnicas semelhantes àquelas desenvolvidas no Exercício 8.11, escreva um programa que produza limericks aleatórios. Modificar esse programa para produzir boas limericks é desafiador, mas o resultado é compensador!

8.13 Falso latim. Escreva um programa que converta frases em português para um falso latim. Essa é uma forma de linguagem codificada normalmente usada por diversão. Existem muitas variações nos métodos usados para formar frases em falso latim. Para simplificar, use o seguinte algoritmo:

Para formar uma frase em falso latim a partir de uma frase em português, separe os tokens da frase em palavras usando a função `strtok`. Para traduzir cada palavra do português para uma palavra em falso latim, passe a primeira letra da palavra para o final, e acrescente as letras

‘ei’. Assim, a palavra ‘salto’ se torna ‘altosei’, a palavra ‘para’ se torna ‘arapei’ e a palavra ‘computador’ se torna ‘omputadorcei’. Os espaços entre as palavras devem ser mantidos. Considere o seguinte: a frase em português consiste em palavras separadas por espaços, não existem sinais de pontuação e todas as palavras possuem duas ou mais letras. A função `printLatinWord` deverá exibir cada palavra. [Dica: toda vez que um token for encontrado em uma chamada para `strtok`, passe o ponteiro do token para a função `printLatinWord` e imprima a palavra em falso latim. Nota: Aqui, fornecemos regras simplificadas para converter palavras em falso latim. Para obter regras mais detalhadas e variações, visite <en.wikipedia.org/wiki/Pig_Latin>.]

8.14 Separação de números de telefone em tokens.

Escreva um programa que leia um número de telefone como uma string na forma (55) 5555-5555. O programa deverá usar a função `strtok` para extrair o código de área, os quatro primeiros dígitos do número do telefone e os quatro últimos dígitos do número do telefone como tokens. Os oito dígitos referentes ao número de telefone devem ser concatenados em uma string. O programa deverá converter a string do código de área para `int` e converter a string do número do telefone para `long`. Tanto o código de área quanto o número do telefone deverão ser exibidos na tela.

8.15 Exibição de uma sentença com palavras invertidas. Escreva um programa que receba uma linha de texto, separe os tokens da linha com a função `strtok` e envie os tokens na ordem inversa.

8.16 Procura de substrings. Escreva um programa que receba uma linha de texto e uma string de pesquisa do teclado. Usando a função `strstr`, localize a primeira ocorrência da string de pesquisa na linha de texto e atribua o local à variável `searchPtr` do tipo `char *`. Se a string de pesquisa for encontrada, imprima o restante da linha de texto a partir dessa string. Depois, use `strstr` novamente para localizar a próxima ocorrência da string de pesquisa na linha de texto. Se houver uma segunda ocorrência, imprima o restante da linha de texto a partir da segunda ocorrência. [Dica: a segunda chamada a `strstr` deve conter `searchPtr + 1` como seu primeiro argumento.]

8.17 Total de ocorrências de uma substring. Escreva um programa com base no programa do Exercício 8.16, que recebe várias linhas de texto e uma string de pesquisa, e usa a função `strstr` para determinar o total de ocorrências da string nas linhas de texto. Imprima o resultado.

8.18 Total de ocorrências de um caractere. Escreva um programa que receba várias linhas de texto e um

caractere de pesquisa, e use a função `strchr` para determinar o total de ocorrências do caractere nas linhas de texto.

- 8.19 Total de letras do alfabeto em uma string.** Escreva um programa baseado no programa do Exercício 8.18, que receba várias linhas de texto e use a função `strchr` para determinar o total de ocorrências de cada letra do alfabeto nas linhas de texto. As letras maiúsculas e minúsculas devem ser contadas juntas. Armazene os totais de cada letra em um array e imprima os valores em formato tabular após os totais terem sido calculados.
- 8.20 Total de palavras em uma string.** Escreva um programa que receba várias linhas de texto, e use `strtok` para contar o número total de palavras. Considere que as palavras estejam separadas por espaços ou caracteres de newline.
- 8.21 Ordenação alfabética de uma lista de strings.** Use as funções de comparação de strings, discutidas na Seção 8.6, e as técnicas para classificar arrays, desenvolvida no Capítulo 6, para escrever um programa que coloque uma lista de strings em ordem alfabética. Use os nomes de 10 ou 15 cidades em seu estado como dados para o seu programa.
- 8.22** O gráfico no Apêndice B mostra as representações em código numérico dos caracteres no conjunto de caracteres ASCII. Estude esse gráfico e depois indique se as afirmações a seguir são *verdadeiras* ou *falsas*.
- A letra 'A' vem antes da letra 'B'.
 - O dígito '9' vem antes do dígito '0'.
 - Os símbolos normalmente usados para representar adição, subtração, multiplicação e divisão vêm antes de qualquer um dos dígitos.
 - Os dígitos vêm antes das letras.
 - Se um programa de classificação ordena strings em uma sequência crescente, então o programa colocará o símbolo de um parêntese direito antes do símbolo de um parêntese esquerdo.
- 8.23 Strings que começam com 'b'.** Escreva um programa que leia uma série de strings e imprima apenas aquelas que comecem com a letra 'b'.
- 8.24 Strings que terminam em 'ed'.** Escreva um programa

que leia uma série de strings e imprima apenas aquelas que terminem com as letras 'ed'.

- 8.25 Impressão de letras para vários códigos ASCII.** Escreva um programa que receba um código ASCII e imprima o caractere correspondente. Modifique esse programa de modo que ele gerencie todos os códigos de três dígitos possíveis que ocorram no intervalo de 000 a 255, e tente imprimir os caracteres correspondentes. O que acontece quando esse programa é executado?
- 8.26 Escreva suas próprias funções de tratamento de caracteres.** Usando o quadro de caracteres ASCII do Apêndice B como guia, escreva suas próprias versões das funções de tratamento de caracteres da Figura 8.1.
- 8.27 Escreva suas próprias funções de conversão de strings.** Escreva suas próprias versões das funções na Figura 8.5 para converter strings em números.
- 8.28 Escreva suas próprias funções de cópia e concatenação de strings.** Escreva duas versões de cada uma das funções de cópia e de concatenação de strings da Figura 8.17. A primeira versão deverá usar subscritos de array, e a segunda, ponteiros e aritmética de ponteiro.
- 8.29 Escreva suas próprias funções de E/S de caractere e de string.** Escreva suas próprias versões das funções `getchar`, `putchar` e `puts` descritas na Figura 8.12.
- 8.30 Escreva suas próprias funções de comparação de strings.** Escreva duas versões da função de comparação de strings da Figura 8.20. A primeira versão deverá usar o subscrito de array, e a segunda, ponteiros e aritmética de ponteiro.
- 8.31 Escreva suas próprias funções de pesquisa de strings.** Escreva suas próprias versões das funções da Figura 8.22 para pesquisa de strings.
- 8.32 Escreva suas próprias funções de tratamento de memória.** Escreva suas próprias versões das funções da Figura 8.30 para manipular blocos de memória.
- 8.33 Escreva suas próprias funções de comprimento de strings.** Escreva duas versões da função `strlen` na Figura 8.36. A primeira versão deverá usar subscritos de array, e a segunda, ponteiros e aritmética de ponteiro.

■ Seção especial: exercícios avançados de manipulação de strings

Os exercícios anteriores estão ligados ao texto e foram criados para testar seu conhecimento sobre os conceitos fundamentais da manipulação de strings. Esta seção inclui uma coleção de problemas de nível intermediário e avançado. São problemas desafiadores, porém, interessantes. Eles variam conside-

ravelmente em dificuldade. Alguns exigirão uma ou duas horas de escrita e implementação do programa. Outros serão úteis para trabalhos de laboratório que poderiam exigir duas ou três semanas, entre estudo e implementação. Alguns são projetos para o final do período.

8.34 *Análise de texto.* A disponibilidade de computadores com capacidades de manipulação de strings resultou em algumas técnicas interessantes de análise de escritas de grandes autores. A dúvida se William Shakespeare realmente existiu tem recebido muita atenção. Alguns estudiosos pensam ter encontrado evidências substanciais que provam que Christopher Marlowe foi quem realmente escreveu as obras-primas atribuídas a Shakespeare. Os pesquisadores usaram computadores para descobrir semelhanças nos escritos desses dois autores. Esse exercício examina três métodos de análise de textos com um computador.

- a) Escreva um programa que leia várias linhas de texto e imprima uma tabela que indique o número de ocorrências de cada letra do alfabeto no texto. Por exemplo, a frase

Ser ou não ser; eis a questão:

contém três 'a', nenhum 'b', nenhum 'c' e assim por diante.

- b) Escreva um programa que leia várias linhas de texto e imprima uma tabela que indique o número de palavras com uma letra, palavras com duas letras, palavras com três letras, e assim por diante, que aparecem no texto. Por exemplo, a frase

Será mais nobre em nosso espírito sofrer.

contém

Tamanho da palavra	Ocorrências
1	0
2	1
3	0
4	2
5	2
6	1
7	0
8	1

- c) Escreva um programa que leia várias linhas de texto e imprima uma tabela que indique o número de ocorrências de cada palavra diferente no texto. A primeira versão de seu programa deverá incluir as palavras na tabela na mesma ordem em que elas aparecerem no texto. Uma saída interessante (e útil) deverá então ser experimentada, uma em que as palavras são ordenadas alfabeticamente. Por exemplo, as linhas

Ser ou não ser: eis a questão:

Será mais nobre em nosso espírito sofrer.

contém as palavras 'ser' duas vezes, a palavra 'ou' uma vez e assim por diante.

8.35 *Processamento de textos.* O tratamento detalhado da manipulação de strings nesse texto é, em grande parte,

atribuída ao crescimento incrível do processamento de textos nos últimos anos. Uma função importante nos sistemas de processamento de textos é a *justificação de texto* — o alinhamento das palavras nas margens esquerda e direita de uma página. Isso gera um documento de aparência profissional, que parece ser composto por tipos em vez de preparado em uma máquina de escrever. A justificação de texto pode ser realizada em sistemas de computador que inserem um ou mais caracteres em branco entre cada uma das palavras em uma linha, de modo que a palavra mais à direita se alinhe com a margem direita.

Escreva um programa que leia várias linhas de texto e o imprima em formato justificado. Considere que o texto deve ser impresso em um papel de 21,59 cm e que margens de 2,54 cm devem ser permitidas nos lados esquerdo e direito da página impressa. Considere que o computador imprima 10 caracteres por polegada horizontal. Portanto, seu programa deverá imprimir 16,51 cm de texto ou 65 caracteres por linha.

8.36 *Impressão de datas em vários formatos.* As datas normalmente são impressas em vários formatos diferentes nas correspondências comerciais. Dois dos formatos mais comuns são

21/07/2011 e 21 de julho de 2011

Escreva um programa que leia uma data no primeiro formato e a imprima no segundo formato.

8.37 *Proteção de cheques.* Os computadores são constantemente usados nos sistemas de escrita de cheque, como aplicações de folha de pagamento e contas a pagar. Circulam muitas histórias sobre cheques de pagamento semanal impressos (por engano) com valores superiores a US\$ 1 milhão. Valores estranhos são impressos por sistemas computadorizados de escrita de cheque por erro humano e/ou falha de máquina. Os projetistas de sistemas, naturalmente, fazem todos os esforços para montar controles em seus sistemas que impeçam que cheques com valores errados sejam emitidos.

Outro problema sério é a alteração intencional de um valor do cheque por alguém com intenções de sacá-lo fraudulentamente. Para impedir que um valor monetário seja alterado, a maioria dos sistemas computadorizados de escrita de cheque emprega uma técnica chamada *proteção de cheque*.

Os cheques planejados para impressão por computador contêm um número fixo de espaços, em que o computador deve imprimir um valor. Suponha que um cheque de pagamento contenha nove espaços em branco nos quais o computador deverá imprimir o valor de um cheque semanal. Se o valor for grande, então os nove espaços serão preenchidos — por exemplo:

```

11.230,60 (valor do cheque)
.....
123456789 (números de posição)

```

Por outro lado, se o valor for menor do que mil, então vários espaços ficarão em branco — por exemplo,

```

    99,87
.....
123456789

```

contém quatro espaços em branco. Um cheque impresso com espaços em branco facilita o trabalho de alguém que queira alterar seu valor. Para impedir que um cheque seja alterado, muitos sistemas de escrita de cheque inserem *asteriscos iniciais* para proteger o valor, da seguinte forma:

```

****99,87
.....
123456789

```

Escreva um programa que receba um valor monetário a ser impresso em um cheque e depois imprima o valor no formato de proteção de cheque com os asteriscos iniciais, se necessário. Suponha que nove espaços estejam disponíveis para a impressão de um valor.

8.38 Escrita por extenso do equivalente de um valor de cheque. Continuando a discussão do exemplo anterior, reiteramos a importância do projeto de sistemas de escrita de cheque para impedir a alteração de valores. Um método de segurança comum exige que o valor do cheque seja escrito em números e ‘por extenso’. Mesmo que alguém seja capaz de alterar o valor numérico do cheque, é extremamente difícil mudar o valor em palavras.

Muitos sistemas computadorizados de escrita de cheque não imprimem o valor do cheque em palavras. Talvez, o principal motivo para essa omissão seja o fato de que a maioria das linguagens de alto nível usadas em aplicações comerciais não contém recursos adequados para manipulação de strings. Outro motivo é que a lógica para escrever equivalentes em texto dos valores do cheque é um pouco complicada.

Escreva um programa que receba um valor numérico de cheque e escreva o texto equivalente ao valor. Por exemplo, o valor 112,43 deve ser escrito como
CENTO E DOZE REAIS E QUARENTA E TRÊS CENTAVOS

8.39 Código Morse. Talvez o mais famoso de todos os esquemas de codificação seja o código Morse, desenvolvido por Samuel Morse em 1832 para uso conjunto com o sistema de telégrafo. O código Morse atribui uma série de pontos e traços para cada letra do alfabeto, cada dígito e alguns caracteres especiais (como ponto, vírgula, dois-pontos e ponto e vírgula). Em sistemas orientados por som, o ponto representa um som curto e o traço representa um som longo. Outras representações dos pontos e traços

são usadas com sistemas orientados a luz e sistemas de bandeira de sinalização.

A separação entre palavras é indicada por um espaço — simplesmente, a ausência de um ponto ou um traço. Em um sistema orientado por som, um espaço é indicado por um curto período de tempo durante o qual nenhum som é transmitido. A versão internacional do código Morse aparece na Figura 8.39.

Caractere	Código	Caractere	Código	Caractere	Código
A	.-	N	-.	Dígitos	
B	-...	O	---	I	..---
C	-.-.	P	.-.-	2	..---
D	-. .	Q	--.-	3	...--
E	.	R	.-.	4	-
F	..-.	S	...	5	
G	--.	T	-	6	-....
H		U	..-	7	--...
I	..	V	...-	8	---..
J	.----	W	.-.-	9	----.
K	-. -	X	-.-.-	0	-----
L	.-...	Y	-. -.-		
M	--	Z	--..		

Figura 8.39 ■ As letras do alfabeto expressas em código Morse internacional.

Escreva um programa que leia uma frase em português e codifique-a em código Morse. Escreva também um programa que leia uma frase em código Morse e a converta para o equivalente em português. Use um espaço entre cada letra do código Morse e três espaços entre cada palavra em código Morse.

8.40 Um programa de conversão métrica. Escreva um programa que ajude o usuário com as conversões métricas. Seu programa deverá permitir que o usuário especifique os nomes das unidades como strings (ou seja, centímetros, litros, gramas, e assim por diante, para o sistema métrico, e polegadas, quartos, libras, e assim por diante, para o sistema inglês) e deverá responder a perguntas simples como

“Quantas polegadas há em 2 metros?”

“Quantos litros há em 10 quartos?”

Seu programa deverá reconhecer conversões inválidas. Por exemplo, a pergunta

“Quantos pés há em 5 quilogramas?”

não faz sentido, pois ‘pés’ são unidades de comprimento, enquanto ‘quilogramas’ são unidades de massa.

8.41 Cartas de cobrança. Muitas empresas gastam muito tempo e dinheiro coletando débitos devidos. A *cobrança* é o processo de fazer pedidos repetidos e insistentes a um devedor em uma tentativa de coletar um débito. Os computadores normalmente são usados para gerar cartas de cobrança automaticamente e em graus de rigidez cada vez maiores, à medida que o tempo da dívida aumenta. A teoria é que, quando um débito está vencido há mais tempo, fica mais difícil receber e, portanto, as cartas de cobrança precisam se tornar mais ameaçadoras. Escreva um programa que contenha os textos de cinco cartas de cobrança de rigidez cada vez maior. Seu programa deverá aceitar como entrada o seguinte:

- a) Nome do devedor.
- b) Endereço do devedor.
- c) Conta do devedor.
- d) Valor devido.
- e) Tempo da dívida (ou seja, um mês de atraso, dois meses de atraso etc.).

Use o tempo da dívida para selecionar um dos cinco textos de mensagem e depois imprima a carta de cobrança, inserindo as outras informações fornecidas onde for apropriado.

■ Um projeto desafiador de manipulação de strings

8.42 Um gerador de palavras cruzadas. A maioria das pessoas já tentou resolver um jogo de palavras cruzadas pelo menos uma vez, mas poucas tentaram criar um. A criação de um diagrama de palavras cruzadas é um problema difícil. Ele é sugerido aqui como um projeto de manipulação de string que exige sofisticação e esforço substanciais. Existem muitos problemas que você precisa resolver para fazer funcionar até mesmo o mais simples programa gerador de palavras cruzadas. Por exemplo, como se representa a grade de um diagrama de palavras cruzadas dentro do computador? Deve-se usar uma série

de strings, ou arrays de subscrito duplo? Você precisa de uma fonte de palavras (ou seja, um dicionário computadorizado) que possa ser referenciado diretamente por um programa. Em que forma essas palavras devem ser armazenadas para facilitar as manipulações complexas exigidas pelo programa? O leitor realmente ambicioso desejará gerar a parte das ‘dicas’ das palavras cruzadas em que as informações breves para cada palavra na ‘horizontal’ e cada palavra na ‘vertical’ sejam impressas para o usuário do diagrama. A mera impressão de uma versão do próprio diagrama em branco não é um problema simples.

■ Fazendo a diferença

8.43 Cozinhar com ingredientes mais saudáveis. A obesidade na América do Norte vem aumentando em uma taxa alarmante. Verifique o mapa do Centers for Disease Control and Prevention (CDC) em www.cdc.gov/nccdphp/dnpa/Obesity/trend/maps/index.htm, que mostra as tendências da obesidade nos Estados Unidos nos últimos vinte anos. À medida que a obesidade aumenta, aumentam as ocorrências de problemas relacionados a ela (por exemplo, doenças do coração, pressão alta, colesterol alto, diabetes tipo 2). Escreva um programa que ajude os usuários a escolher ingredientes mais saudáveis ao cozinhar, e ajude os alérgicos a certos alimentos (por exemplo, castanhas, glúten) a encontrar substitutos. O programa deverá ler uma receita do usuário e sugerir substitutos mais saudáveis para alguns dos ingredientes. Para simplificar, seu programa deverá considerar que a receita não tem abreviações para medidas

como colheres de chá, xícaras e colheres de sobremesa, e usa dígitos numéricos para quantidades (por exemplo, 1 ovo, 2 xícaras) em vez de escrevê-los por extenso (um ovo, duas xícaras). Algumas substituições comuns aparecem na Figura 8.40. Seu programa deverá exibir um aviso como ‘Sempre consulte seu médico antes de fazer mudanças significativas em sua dieta’.

Seu programa deverá levar em consideração que as substituições nem sempre são um para um. Por exemplo, se uma receita de bolo pedir três ovos, pode ser razoável usar seis claras de ovos em seu lugar. Os dados de conversão para medidas e substitutos podem ser obtidos em sites como:

```
<www.cems.com.br/nutricao/?p=296>
<www.nutricaoativa.com.br/arquivos/tabela1.pdf>
<http://tiojuliao.diabetes.org.br/Bet_
Balanca/Dias_de_nutricao/nut31.php>
```

Seu programa deverá considerar as questões de saúde do usuário, como colesterol, pressão sanguínea, perda de peso, alergias e assim por diante. Para colesterol alto, o programa deverá gerar substitutos para ovos e laticínios; se o usuário deseja perder peso, substitutos com baixa caloria para ingredientes como açúcar devem ser sugeridos.

Ingrediente	Substituição
1 xícara de creme de leite	1 xícara de iogurte
1 xícara de leite	1/2 xícara de leite condensado e 1/2 xícara de água
1 colher de sobremesa de suco de limão	1/2 colher de sobremesa de vinagre
1 xícara de açúcar	1/2 xícara de mel, 1 xícara de melado ou 1/4 de xícara de adoçante
1 xícara de manteiga	1 xícara de margarina ou iogurte
1 xícara de farinha de trigo	1 xícara de farinha integral ou de arroz
1 xícara de maionese	1 xícara de queijo tipo cottage ou 1/8 de xícara de maionese e 7/8 de xícara de iogurte
1 ovo	2 colheres de sobremesa de amido de milho, farinha de mandioca ou amido de batata ou 2 claras de ovos ou metade de uma banana grande (amassada)
1 xícara de leite	1 xícara de leite de soja
1/4 xícara de azeite	1/4 de xícara de molho de maçã
pão branco	pão integral

Figura 8.40 ■ Substitutos típicos dos ingredientes.

8.44 Analisador de spam. O spam (ou e-mail não solicitado) custa às organizações dos Estados Unidos bilhões de dólares por ano em software para prevenção de spam, equipamento, recursos de rede, largura de banda e produtividade perdida. Pesquise on-line algumas das mensagens e palavras de spam que são comumente enviadas por e-mail, e verifique sua própria pasta de caixa de e-mail. Crie uma lista com trinta palavras e frases normalmente encontradas nas mensagens de spam. Escreva um programa em que o usuário forneça via teclado uma mensagem de e-mail. Leia a mensagem em um array de caracteres

grande e garanta que o programa não tente inserir caracteres após o final do array. Depois, filtre a mensagem para cada uma das trinta palavras-chave ou frases. Para cada ocorrência de uma delas, some um ponto no 'nível de spam' da mensagem. Em seguida, avalie a probabilidade de que a mensagem seja um spam com base no número de pontos que ela recebeu.

8.45 Linguagem SMS. Short Message Service (SMS) é um serviço de comunicações que permite o envio de mensagens de texto de 160 ou menos caracteres entre telefones móveis. Com a proliferação do uso mundial do telefone móvel, SMS está sendo usado em muitas nações em desenvolvimento para propósitos políticos (por exemplo, para expressar opiniões e oposição), para informar sobre desastres naturais e assim por diante. Acesse <comunica.org/radio2.0/archives/87> para ver exemplos disso. Como o comprimento das mensagens de SMS é limitado, normalmente é usada a linguagem SMS — abreviações de palavras e frases comuns nas mensagens de texto móveis, e-mails, mensagens instantâneas etc. Por exemplo, 'EMO' quer dizer 'em minha opinião' na linguagem SMS. Procure informações sobre a linguagem SMS on-line. Escreva um programa em que o usuário possa digitar uma mensagem usando a linguagem SMS, e que depois possa traduzi-la para o português. Ofereça também um mecanismo para traduzir o texto escrito em português para a linguagem SMS. Um problema potencial é que uma abreviação em SMS pode também ter outros significados. Por exemplo, EMO (conforme usado acima) também poderia significar 'Empresa Marítima Ocidental' etc.

8.46 Neutralidade de gênero. No Exercício 1.12, você pesquisou a eliminação do gênero em todas as formas de comunicação. Depois, você descreveu o algoritmo que usaria para ler um parágrafo de texto e substituir palavras de gênero específico por equivalentes sem gênero. Crie um programa que leia um parágrafo de texto, e que depois substitua palavras de gênero específico por outras sem gênero. Apresente o texto resultante sem gênero.